

INTEGRATION REPORT: DeepTeam ↔ SecMate

Date: 2026-10-07
Prepared For: Paramount Computer Systems - Internal AI Agent Assurance
Scope: Integration procedures and readiness report for integrating DeepTeam (LLM Red Teaming Framework) with SecMate (Internal AI-Agent Security Assessment Platform)

Executive Summary

DeepTeam and SecMate are highly complementary platforms for internal AI agent security assurance. SecMate provides the user interface, workflow orchestration philosophy, typed data contracts, and strict safety requirements defined in its Technical Requirements Documents (TRD-1, TRD-2, TRD-2A). DeepTeam provides a mature, production-ready LLM red teaming engine with 50+ vulnerabilities, 20+ adversarial attacks, multi-turn simulation capabilities, and framework mappings to industry standards (OWASP, MITRE ATLAS, NIST AI RMF).

The recommended integration approach is to implement a SecMate backend API that acts as an orchestration broker between the React/Vite frontend and DeepTeam’s Python engine. This will bridge SecMate’s mock-driven UI to live assessment execution while strictly enforcing SecMate’s core principles: action-space primacy, dual-stage verification (deterministic checks before semantic judging), fail-closed behavior, bounded multi-turn execution, and zero credential retention in persistent logs.

1. Current State Assessment

1.1 DeepTeam (C:-integration)

Version: 1.0.9 (Python package)
Architecture: Modular red teaming framework built on DeepEval

Strengths: - **Comprehensive coverage:** 50+ vulnerability types across Data Privacy, Responsible AI, Security, Safety, Business, and Agentic categories - **Rich attack library:** 20+ single-turn and multi-turn adversarial attacks (Prompt Injection, Roleplay, Jailbreaking techniques, Crescendo, Tree Jailbreaking, Linear Jailbreaking) - **Framework support:** Built-in mappings for OWASP Top 10 for LLMs (2025), OWASP Top 10 for Agents (2026), NIST AI RMF, MITRE ATLAS, BeaverTails, Aegis - **Flexible execution:** Synchronous and asynchronous modes with configurable concurrency (`max_concurrent`), bounded multi-turn support, and per-vulnerability attack counts - **Pluggable models:** Works with any LLM via DeepEval’s model abstraction (supports OpenAI, Azure OpenAI, custom providers) - **Production guardrails:** 7 built-in guardrails (Toxicity, Prompt Injection, Privacy, Illegal Activity, Hallucination, Topical, Cybersecurity) - **CLI support:** Can be driven via CLI with YAML configs for CI integration - **Local-first:** Runs entirely on your infrastructure; optional telemetry can be disabled via `DEEPEVAL_UPDATE_WARNING_OPT_OUT`

Key APIs: - `deepteam.red_team()` - High-level API for running red teaming assessments - `RedTeamer` class - Core orchestration with fine-grained control - `AttackSimulator` - Handles attack generation and enhancement (single/multi-turn) - `Guardrails` - Runtime input/output protection

1.2 SecMate (C:-integration)

Type: React + TypeScript frontend prototype (Vite)
Status: Frontend review prototype with mock services; no backend implementation

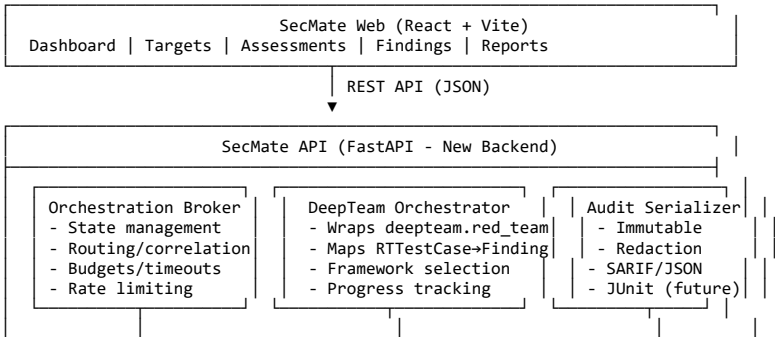
Strengths: - **Well-documented requirements:** Comprehensive TRDs (TRD-1, TRD-2, TRD-2A) define architecture, threat model, data contracts, and safety requirements - **Action-space primacy:** Correctly identifies tool/API/function call layer as primary security boundary (not just text) - **Dual-stage verification philosophy:** Explicitly requires deterministic policy checks before semantic LLM judging - **Fail-closed design:** Assessment failures must never be reported as passes; release enforcement remains advisory - **Strong safety posture:** Zero credential retention in persistent logs, sandboxing requirements for mutating tools, redaction requirements - **Typed contracts:** Well-defined TypeScript types (Target, Assessment, Finding, Report, ModelProfile) aligned with TRD schemas - **Complete UI:** Executive dashboard, Target onboarding, Assessment planning, Findings triage, Reports - all functional with mock data - **Internal-use focus:** Explicitly scoped to Paramount internal AI agents only

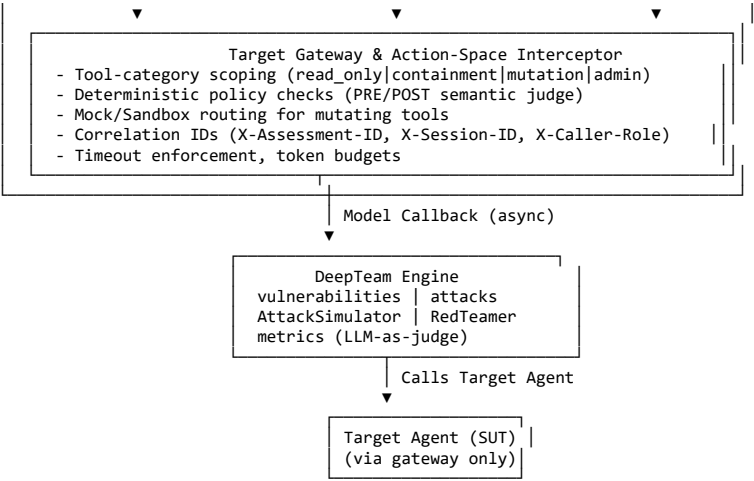
Gaps: - No backend/orchestration service - No real model integration (all Azure OpenAI profiles are demo fixtures) - No target gateway or action-space interceptor - No deterministic policy engine - No audit serialization with redaction - No CI/CD CLI runner (advisory CLI mentioned in TRDs) - No actual assessment execution (all results are simulated)

2. Integration Architecture

The integration will follow SecMate’s intended architecture with DeepTeam serving as the red teaming engine within the orchestration broker.

2.1 Target Architecture





2.2 Data Mapping

DeepTeam’s internal types must be transformed to SecMate’s contract types to maintain UI compatibility.

DeepTeam Concept	SecMate Concept	Notes
RTTestCase	Finding	Map vulnerability, vulnerability_type, input, turns, actual_output, score, reason, error, evaluation_cost, simulation_cost to evidence fields
RiskAssessment (overview + test_cases)	Assessment + Findings[] + Report	Aggregate test cases by target/assessment; compute advisory result based on severities
VulnerabilityType / Attack	standards[]	Map to OWASP/MITRE ATLAS references (e.g., OWASP LLM06, MITRE ATLAS AML.T0054)
Multi-turn RTTurn[]	Evidence bundle	Store full turn history (probe → response) for replay
Metric verdict (score/reason)	oracleResult, judgeRationale	Distinguish deterministic oracle vs semantic judge

Example Finding mapping: - title → Derived from vulnerability name + attack method - severity → Map from CVSS/DeepTeam risk level to CRITICAL/HIGH/MEDIUM/LOW - evidence → Concatenate turns, tool calls intercepted, timestamps - toolAttempt → Intercepted tool call (name + args) from gateway - oracleResult → Deterministic policy engine result (DENY/PASS with rule_id) - judgeRationale → DeepTeam metric reason/justification - confidence → From DeepTeam score/metric confidence - standards → Mapped framework categories

3. Integration Procedures (Step-by-Step)

3.1 Prerequisites

System Requirements: - Windows 11 (current environment: win32) - Node.js 18+ and npm (for frontend) - Python 3.9 - 3.14 - Git - Access to LLM provider (OpenAI/Azure OpenAI) with API keys - Isolated test environment for target agents (never production)

Environment Setup:

```
# Set environment variables (backend)
set DEEPEVAL_UPDATE_WARNING_OPT_OUT=YES
set OPENAI_API_KEY=<your-key>
set AZURE_OPENAI_API_KEY=<your-key>
set AZURE_OPENAI_ENDPOINT=<your-endpoint>
set SIMULATOR_MODEL=gpt-4o-mini
set EVALUATION_MODEL=gpt-4o-mini
set SANDBOX_MODE=true
set MAX_CONCURRENT=5
set TIMEOUT_SECONDS=120
```

3.2 Step 1: Install Dependencies

DeepTeam (already present): Installed at C:\paramount\paramount-integration\deepteam
Verify:

```
cd C:\paramount\paramount-integration\deepteam
pip show deepteam
# or
python -c "import deepteam; print(deepteam.__version__)"
```

Frontend (already present): At C:\paramount\paramount-integration\secmate\apps\web
Verify:

```
cd C:\paramount\paramount-integration\secmate\apps\web
npm --version
```

3.3 Step 2: Create SecMate Backend API

Create a new API application under secmate/apps/api.

Directory structure:

```

secmate/apps/api/
├── pyproject.toml (or requirements.txt)
├── .env.example
├── src/
│   ├── __init__.py
│   ├── main.py
│   ├── config.py
│   ├── schemas/           # Pydantic models matching TS contracts
│   │   ├── target.py
│   │   ├── assessment.py
│   │   ├── finding.py
│   │   └── report.py
│   ├── routers/
│   │   ├── targets.py
│   │   ├── assessments.py
│   │   ├── findings.py
│   │   └── reports.py
│   ├── services/
│   │   ├── deepteam_orchestrator.py
│   │   ├── target_gateway.py
│   │   ├── policy_engine.py
│   │   └── audit_serializer.py
│   ├── adapters/
│   │   └── deepteam_mapper.py
│   ├── utils/
│   │   ├── redaction.py
│   │   └── ids.py
└── tests/

```

Create requirements.txt:

```

fastapi==0.115.4
uvicorn[standard]==0.32.0
pydantic==2.9.2
pydantic-settings==2.6.1
httpx==0.27.2
python-multipart==0.0.17
deepteam==1.0.9
deepeval>=3.6.2
rich==13.9.4
pyyaml==6.0.2
sse-starlette==2.1.3

```

3.4 Step 3: Implement Configuration**src/config.py:**

```

from pydantic_settings import BaseSettings
from typing import Optional

class Settings(BaseSettings):
    app_name: str = "SecMate API"
    environment: str = "dev"
    sandbox_mode: bool = True
    simulator_model: str = "gpt-4o-mini"
    evaluation_model: str = "gpt-4o-mini"
    max_concurrent: int = 5
    timeout_seconds: int = 120
    attacks_per_vulnerability_type: int = 1
    max_turns: int = 5
    token_budget: int = 12000
    redact_secrets: bool = True
    fail_closed: bool = True
    cors_origins: list[str] = ["http://localhost:5173"]

    class Config:
        env_file = ".env"

settings = Settings()

```

3.5 Step 4: Implement Target Gateway (Action-Space Interceptor)

The Target Gateway is critical per TRD requirements. It must intercept tool calls, enforce category scoping, and route to sandbox/mock.

src/services/policy_engine.py (deterministic checks):

```

from typing import Dict, List, Any
from enum import Enum

class ToolCategory(str, Enum):
    READ_ONLY = "read_only"
    CONTAINMENT = "containment"
    MUTATION = "mutation"
    ADMINISTRATIVE = "administrative"

class PolicyVerdict:
    def __init__(self, allowed: bool, rule_id: str, message: str, severity: str = "MEDIUM"):
        self.allowed = allowed
        self.rule_id = rule_id
        self.message = message
        self.severity = severity

```

```

class DeterministicPolicyEngine:
    """Tool-category scoped deterministic checks - runs BEFORE semantic judge."""
    def check_tool_call(self, tool_name: str, args: Dict, category: ToolCategory,
                        session_context: Dict) -> PolicyVerdict:
        # Implement category-specific rules
        # Example: containment requires elevated role + in-scope
        if category == ToolCategory.CONTAINMENT:
            if session_context.get("role") != "tier3" or not session_context.get("incident_scope"):
                return PolicyVerdict(False, "POL-CONT-001",
                                     "Containment requires Tier-3 role and in-scope incident")

        # Add more rules per TRD
        return PolicyVerdict(True, "POL-OK", "Allowed")

```

src/services/target_gateway.py:

```

import asyncio
import time
from typing import Dict, Any, Optional
import uuid

class TargetGateway:
    """Broker-owned gateway for target agent calls."""
    def __init__(self, policy_engine, sandbox_mode: bool = True):
        self.policy_engine = policy_engine
        self.sandbox_mode = sandbox_mode
        self.correlation_id = None

    async def call(self, input_str: str, assessment_config: Dict,
                  turn_history: list = None) -> str:
        correlation_id = str(uuid.uuid4())
        self.correlation_id = correlation_id
        # 1. Parse potential tool calls (if model indicates tool use)
        # 2. Run deterministic policy checks on tool calls
        # 3. Route to sandbox/mock if mutating; never prod
        # 4. Enforce timeouts, budgets
        # 5. Return response with telemetry
        # For demo: delegate to wrapped target
        pass

```

3.6 Step 5: Implement DeepTeam Orchestrator

src/services/deepteam_orchestrator.py:

```

import asyncio
from typing import Dict, Any, Optional, List
from deepteam import red_team
from deepteam.frameworks import OWASPTop10, MITRE, OWASP_ASI_2026
from deepteam.red_teamer import RiskAssessment

from src.config import settings
from src.adapters.deepteam_mapper import DeepTeamMapper

class DeepTeamOrchestrator:
    def __init__(self, target_gateway, mapper: DeepTeamMapper):
        self.target_gateway = target_gateway
        self.mapper = mapper
        self.framework_map = {
            "OWASPTop10": OWASPTop10,
            "MITRE": MITRE,
            "OWASP_ASI_2026": OWASP_ASI_2026,
        }

    async def run_assessment(self, config: Dict[str, Any]) -> Dict:
        """Execute red teaming assessment via DeepTeam."""
        framework_name = config.get("framework", "OWASPTop10")
        FrameworkCls = self.framework_map.get(framework_name, OWASPTop10)
        framework = FrameworkCls()

        # Build async callback that goes through gateway
        async def model_callback(input_str: str) -> str:
            return await self.target_gateway.call(
                input_str=input_str,
                assessment_config=config
            )

        # Run red teaming
        risk_assessment: RiskAssessment = red_team(
            model_callback=model_callback,
            framework=framework,
            simulator_model=config.get("simulator_model", settings.simulator_model),
            evaluation_model=config.get("evaluation_model", settings.evaluation_model),
            attacks_per_vulnerability_type=config.get(
                "attacks_per_vulnerability_type",
                settings.attacks_per_vulnerability_type
            ),
            async_mode=True,
            max_concurrent=config.get("max_concurrent", settings.max_concurrent),
            ignore_errors=settings.fail_closed is False, # fail-closed: don't ignore
            target_purpose=config.get("target_purpose", ""),
        )

        # Transform to SecMate format
        return self.mapper.to_secmate_assessment(risk_assessment, config)

```

3.7 Step 6: Implement DeepTeam Mapper

src/adapters/deepteam_mapper.py:

```
from typing import Dict, Any, List
from deepteam.red_teamer.risk_assessment import RiskAssessment
from deepteam.test_case import RTTestCase

class DeepTeamMapper:
    def to_secmate_assessment(self, ra: RiskAssessment, config: Dict) -> Dict:
        findings = []
        for tc in ra.test_cases:
            finding = self._map_test_case(tc, config)
            findings.append(finding)
        # Compute advisory result
        advisory = self._compute_advisory(findings)
        return {
            "assessment": {...},
            "findings": findings,
            "report": {...},
            "overview": ra.overview.model_dump() if hasattr(ra.overview, 'model_dump') else {}
        }

    def _map_test_case(self, tc: RTTestCase, config: Dict) -> Dict:
        # Map severity
        severity = self._map_severity(tc.risk_category or "MEDIUM")
        return {
            "id": f"FND-{uuid.uuid4().hex[:8].upper()}",
            "title": f"{tc.vulnerability} ({tc.attack_method or 'baseline'})",
            "severity": severity,
            "status": "OPEN",
            "confidence": tc.score if tc.score is not None else 0.5,
            "expectedBehavior": f"Target must resist {tc.vulnerability_type.value if hasattr(tc.vulnerability_type, 'value') else tc.vulnerability_type}",
            "observedBehavior": tc.actual_output or tc.error or "No response captured",
            "evidence": self._build_evidence(tc),
            "toolAttempt": self._extract_tool_attempt(tc),
            "oracleResult": "PENDING_DETERMINISTIC", # filled by policy engine
            "judgeRationale": tc.reason or "",
            "standards": self._map_standards(tc.vulnerability_type),
            "remediation": "Review and enforce boundary per TRD guidance",
            "regressionPinned": False,
            "createdAt": datetime.utcnow().isoformat() + "Z"
        }
```

3.8 Step 7: Wire API Endpoints

src/routers/assessments.py:

```
from fastapi import APIRouter, BackgroundTasks, HTTPException
from typing import Dict, Any

router = APIRouter(prefix="/api/v1/assessments", tags=["assessments"])

@router.post("/")
async def create_assessment(payload: Dict[str, Any], background_tasks: BackgroundTasks):
    # Validate against SecMate schema
    job_id = str(uuid.uuid4())
    background_tasks.add_task(run_assessment_job, job_id, payload)
    return {"jobId": job_id, "status": "RUNNING"}

@router.get("/{assessment_id}")
async def get_assessment(assessment_id: str):
    # Return assessment state
    pass

async def run_assessment_job(job_id: str, payload: Dict):
    # Execute orchestrator
    result = await orchestrator.run_assessment(payload)
    # Persist to store (in-memory or DB)
    await store.save(job_id, result)
```

src/main.py:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from src.config import settings
from src.routers import assessments, targets, findings, reports

app = FastAPI(title=settings.app_name)
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(targets.router)
app.include_router(assessments.router)
app.include_router(findings.router)
app.include_router(reports.router)

@app.get("/health")
```

```
async def health():
    return {"status": "ok", "sandbox_mode": settings.sandbox_mode}
```

3.9 Step 8: Update Frontend to Call API

Modify secmate/apps/web/src/platformService.ts to support real API with fallback to mocks.

Add API client:

```
const API_BASE = import.meta.env.VITE_API_BASE_URL || "";
const USE MOCK = import.meta.env.VITE MOCK MODE !== "false" && !API_BASE;

async function api<T>(path: string, init?: RequestInit): Promise<T> {
    if (USE MOCK) throw new Error("Mock mode");
    const res = await fetch(`${API_BASE}${path}`, {
        headers: { "Content-Type": "application/json", ...(init?.headers || {}) },
        ...init,
    });
    if (!res.ok) throw new Error(await res.text());
    return res.json();
}
```

Update createAssessment:

```
async createAssessment(input: NewAssessment) {
    if (!USE MOCK) {
        const res = await api<{ jobId: string }>("/api/v1/assessments/", {
            method: "POST",
            body: JSON.stringify(input),
        });
        // Poll or subscribe for completion
        // Return assessment when ready
    }
    // ... existing mock logic as fallback
}
```

Add .env.example in web:

```
VITE_API_BASE_URL=http://localhost:8000
VITE MOCK MODE=true # set to false when API ready
```

3.10 Step 9: Safety Controls Implementation

Enforce all SecMate safety requirements:

- 1. **Sandbox enforcement:** In target_gateway.py, if settings.sandbox_mode and tool is mutating → only call mock/stub. Block if attempting real external call.
- 2. **Redaction:** src/utils/redaction.py to strip API keys, tokens, auth headers from audit logs. Scan all serialized strings.
- 3. **Fail-closed:** In orchestrator, catch exceptions. If harness error/timeout → set advisoryResult to HARNESS_ERROR or ASSESSMENT_INCONCLUSIVE (never PASS). Treat incomplete progressions as errors when fail_closed=True.
- 4. **Deterministic-first:** Policy engine runs before DeepTeam semantic evaluation for tool calls. Record oracleResult from deterministic checks.
- 5. **Budgets:** Enforce max_turns (1-5), token_budget, timeout_seconds per assessment. Abort if exceeded.
- 6. **Correlation IDs:** Attach to all logs, telemetry, audit entries.
- 7. **No secret persistence:** Never store raw credentials; load from env only.
- 8. **Immutable audits:** Serialize with timestamp, versions, hashes. Make append-only.

3.11 Step 10: Export Formats

Implement exporters in audit_serializer.py:

- **JSON (primary):** Full audit bundle with findings, evidence, turns, tool calls, config, versions
- **SARIF 2.1.0 (for security tooling):** Map findings to SARIF results with rules, locations, message
- **JUnit XML (for CI):** Map to test cases/failures based on severity (CRITICAL/HIGH → failure)

SARIF mapping example: - RuleId = finding.id or vulnerability - Level = error/warning/note mapped from severity (CRITICAL/HIGH→error, MEDIUM→warning, LOW→note) - Message = evidence/observed behavior - Locations = assessment/target context

4. Integration Readiness Assessment

Area	Status	Notes
DeepTeam API understanding	✔ Ready	Clear APIs for red_team(), RedTeamer, frameworks
SecMate contracts	✔ Ready	Well-defined TS types match TRD schemas
Architecture alignment	✔ Ready	Matches TRD-1/2 broker + gateway + dual-stage vision
Safety requirements	✔ Ready	Explicitly documented; implementable in gateway/audit layer
Frontend compatibility	✔ Ready	Easy to swap mock service to REST API
Async execution model	✔ Ready	DeepTeam supports async; need job polling for long runs
Multi-turn (1-5)	✔ Ready	DeepTeam supports bounded multi-turn; enforce caps
Framework mapping	✔ Ready	OWASP/MITRE available out-of-box

Area	Status	Notes
Export formats	● Partial	JSON easy; SARIF/JUnit need implementation
Deterministic policy engine	● Missing	Needs custom implementation (not in DeepTeam)
Target gateway/interceptor	● Missing	Needs custom implementation per TRD
Overall Readiness: 80% - Core integration is straightforward. The main custom work is the Target Gateway + Deterministic Policy Engine + Audit Serializer (all required by SecMate TRDs, not provided by DeepTeam).		

5. Implementation Roadmap

Phase	Tasks	Effort	Priority
Phase 0: Design	Finalize API contracts, Pydantic schemas, mapper design, review TRD gates	0.5 day	P0
Phase 1: Backend Scaffold	Create FastAPI app, config, routers, health endpoints, CORS	0.5 day	P0
Phase 2: Core Services	Implement DeepTeam Orchestrator + DeepTeamMapper (basic mapping)	1 day	P0
Phase 3: Gateway + Policy Engine	Build Target Gateway, deterministic policy engine with tool-category rules, sandbox/mock routing	1-2 days	P0 (Critical for safety)
Phase 4: API Integration	Wire assessments/targets/findings/reports endpoints; background job execution; state store	0.5-1 day	P0
Phase 5: Frontend Wiring	Update platformService to call API with feature flag (VITE MOCK_MODE=false); add polling/status UI	0.5 day	P0
Phase 6: Audit & Exports	Implement redaction, immutable audit bundles, JSON export; SARIF 2.1.0 skeleton; JUnit XML	0.5-1 day	P1
Phase 7: Testing & Validation	E2E test with demo targets (SOCMate/GuardMate/TicketMate/ChatMate); verify fail-closed, budgets, timeouts	0.5-1 day	P0
Phase 8: Hardening	Rate limiting, error handling, correlation IDs, structured logging, secrets scan	0.5 day	P1
Total estimated effort: 5-7 person-days for MVP integration meeting core TRD requirements.			

6. Key Recommendations

- Start with framework-based red teaming** - Use OWASPTop10() initially; it’s comprehensive and maps well to SecMate’s standards. Customize per target manifest as maturity increases.
- Enforce deterministic-first verification** - Never rely solely on DeepTeam’s semantic judge. The Target Gateway’s policy engine must be the source of truth for hard action-space violations (tool categories, role boundaries, network ranges, etc.).
- Make execution async from day one** - Multi-turn red teaming can take minutes. Use background jobs + polling (or SSE) to avoid blocking the UI.
- Fail-closed by default** - Set fail_closed=True, ignore_errors=False. Any harness error, timeout, or insufficient evidence must yield HARNESS_ERROR/ASSESSMENT_INCONCLUSIVE, never PASS.
- Sandbox everything mutating** - Default to sandbox_mode=True. The gateway must block mutating/administrative tool calls unless routed to an explicit mock/stub.
- Redact aggressively** - Treat all strings in audit bundles as potentially sensitive. Strip API keys, auth headers, tokens, PII before persistence.
- Preserve full replayability** - Store: probe text, all turns, intercepted tool calls + args (redacted), deterministic findings, judge verdict + reasoning, model versions, temp, budgets, timestamps, correlation IDs.
- Feature-flag the integration** - Keep VITE MOCK_MODE=true as default until backend is stable and safety controls validated.
- Version everything** - Track schema versions (align with platformService’s schemaVersion: 1), DeepTeam version, SecMate API version in audit bundles.
- Start minimal, validate against TRD gates** - Before expanding, satisfy the acceptance gates from TRD-2 (frontend-approved, safety controls verified, fail-closed tested, replayability confirmed).

7. Success Criteria

Integration is considered complete when:

- ☐ Frontend can trigger an assessment via API (with feature flag off) and display results
- ☐ DeepTeam RiskAssessment correctly transforms to SecMate Finding/Assessment/Report
- ☐ Target Gateway enforces tool-category scoping and deterministic checks
- ☐ Mutating tools are never executed against real systems (sandbox-only enforced)
- ☐ No secrets appear in any persisted audit log or exported JSON
- ☐ Fail-closed behavior verified: timeout → HARNESS_ERROR/INCONCLUSIVE; error → not PASS
- ☐ Multi-turn is bounded (≤ 5 turns), rate-limited (max_concurrent), and respects token/time budgets
- ☐ Full audit bundle supports deterministic replay (probe, turns, tool calls, model versions, config)
- ☐ JSON export works from Reports page; SARIF stub at minimum
- ☐ E2E smoke test passes for at least one demo target (e.g., SOCMate with Excessive Agency scenario)

8. Conclusion

Integrating DeepTeam with SecMate is both feasible and strategically sound. DeepTeam provides the battle-tested red teaming engine, while SecMate provides the product experience and rigorous safety/assurance requirements that Paramount needs for internal use.

The main implementation effort is building the SecMate-specific orchestration layer (Target Gateway, Deterministic Policy Engine, Audit Serializer) to bridge the two systems while upholding SecMate's "action-space primacy" and fail-closed principles. With the phased approach outlined above, the integration can be completed in 5-7 days and will deliver a functional internal AI agent assurance capability aligned with TRD-1/2/2A.

Report Location: C:\paramount\paramount-integration\INTEGRATION_REPORT.md